TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

MÔN HỌC

ĐIỆN TOÁN ĐÁM MÂY

ĐỀ TÀI:

XÂY DỰNG QUY TRÌNH CI/CD VÀ TRIỂN KHAI HỆ THỐNG NHẬN DIỆN NGÔN NGỮ KÝ HIỆU TIẾNG VIỆT DỰA TRÊN KIẾN TRÚC SERVERLESS CONTAINER TRÊN GOOGLE CLOUD

Giảng viên hướng dẫn: **TS. Nguyễn Đức Bình**

Sinh viên thực hiện: **Trần Phương Đoàn**
Hà Văn Dũng
Dương Vũ Duy
Nguyễn Đức Dương

Lớp: **CNTT K22G**

Thái Nguyên, năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

1. Về tinh thần, thái độ làm việc của nhóm sinh viên:

- Ý thức kỷ luật, thực hiện nội quy học phần: .....
- Tinh thần chủ động, trách nhiệm và khả năng phối hợp nhóm: .....
- Khả năng nghiên cứu, tự học và tiếp cận công nghệ mới: .....

2. Về nội dung báo cáo và kết quả thực hiện đề tài:

- Tính cấp thiết và mức độ đáp ứng mục tiêu môn học Điện toán đám mây: .....
- Chất lượng kiến trúc hệ thống, việc đóng gói container và triển khai Serverless: ..
- Mức độ hoàn thiện của quy trình CI/CD tự động hóa trên GitHub Actions: .....
- Đánh giá chất lượng thực nghiệm, độ trễ và khả năng tối ưu chi phí $0: .....

3. Về hình thức trình bày:

- Bố cục quyển báo cáo, tính khoa học và tuân thủ quy chuẩn định dạng ICTU: ....

4. Điểm đánh giá và kết luận:

- Điểm số (bằng số): ..... (bằng chữ: .....)
- Ý kiến đồng ý cho bảo vệ/nghiệm thu: .....

Thái Nguyên, ngày ..... tháng ..... năm 2026

GIẢNG VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

LỜI CẢM ƠN

Để hoàn thành bài tập lớn môn **Điện toán đám mây** với đề tài *"Xây dựng quy trình CI/CD và triển khai hệ thống nhận diện ngôn ngữ ký hiệu tiếng Việt dựa trên kiến trúc Serverless Container trên Google Cloud"*, nhóm 2 chúng em xin bày tỏ lòng biết ơn sâu sắc nhất tới các thầy cô giáo trường Đại học Công nghệ Thông tin và Truyền thông (ICTU) — Đại học Thái Nguyên.

Đặc biệt, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới thầy **ThS. Nguyễn Đức Bình**, giảng viên trực tiếp giảng dạy và hướng dẫn học phần. Thầy đã truyền đạt cho chúng em những nền tảng kiến thức thực tế quý báu về kiến trúc điện toán đám mây, các giải pháp Serverless, công nghệ container hóa và tư duy văn hóa DevOps. Những định hướng, góp ý tận tình của thầy trong suốt quá trình triển khai là nguồn động lực to lớn giúp nhóm định hình và hoàn thiện hệ thống này.

Mặc dù các thành viên trong nhóm đã nỗ lực hết mình để hiện thực hóa một quy trình CI/CD và kiến trúc đám mây chuẩn mực, song do kinh nghiệm thực tế còn hạn chế nên bài báo cáo chắc chắn không tránh khỏi những thiếu sót. Nhóm 2 rất mong nhận được những lời nhận xét, chỉ dẫn quý báu từ thầy giáo và hội đồng nghiệm thu để đề tài được hoàn thiện tốt hơn.

Chúng em xin kính chúc thầy cô luôn dồi dào sức khỏe, hạnh phúc và thành công trên con đường sự nghiệp trồng người!

Thái Nguyên, tháng 10 năm 2026

Thành viên Nhóm 2

MỤC LỤC

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN i

LỜI CẢM ƠN ii

MỤC LỤC iii

DANH MỤC TỪ VIẾT TẮT vi

DANH MỤC BẢNG BIỂU vii

DANH MỤC HÌNH VẼ viii

LỜI MỞ ĐẦU 1

1 TỔNG QUAN VỀ BÀI TOÁN VÀ CƠ SỞ KỸ THUẬT 3

1.1 Bối cảnh bài toán và các công nghệ sử dụng 3

1.1.1 Nguồn gốc và mục tiêu đề tài 3

1.1.2 Các công nghệ cốt lõi và vai trò trong dự án 3

1.2 Hiện trạng mô hình AI gốc từ Đồ án Capstone SIC 5

1.2.1 Đặc tả bài toán và bảng chữ cái VSL (23 ký tự) 5

1.2.2 Đường ống xử lý AI và kiến trúc ResNet50 5

1.2.3 Các hạn chế thực tế của hệ thống ban đầu 7

1.3 Định hình giải pháp nâng cấp hệ thống 7

1.3.1 Các yêu cầu cốt lõi của hệ thống nâng cấp 7

1.3.2 Kiến trúc phân tách Decoupled và tối ưu chi phí $0 9

1.4 Tóm tắt chương 1 10

2 THIẾT KẾ GIẢI PHÁP VÀ XÂY DỰNG HỆ THỐNG 11

2.1 Tái cấu trúc mã nguồn AI và xây dựng dịch vụ RESTful API 11

2.1.1 Chuẩn hóa module trích xuất MediaPipe và thuật toán Gaussian Heatmap 11

2.1.2 Độc lập hóa cấu hình và nguồn chân lý nhãn theo Twelve-Factor App 11

2.1.3 Xây dựng dịch vụ FastAPI với các endpoint kiểm tra sức khỏe và suy luận 11

2.2 Đóng gói Container với Docker 11

2.2.1 Chiến lược lựa chọn Base Image tối ưu kích thước cho môi trường Linux 11

2.2.2 Xử lý phụ thuộc thư viện Deep Learning và OpenCV Headless . 11

2.2.3 Cấu hình Dockerfile tối ưu đa tầng và thiết lập bảo mật tiến trình 11

2.3 Thiết kế kiến trúc hạ tầng Cloud Serverless 11

2.3.1 Cấu hình dịch vụ Backend trên Google Cloud Run 11

2.3.2 Triển khai dịch vụ Frontend trên Cloudflare Pages 11

2.4 Xây dựng quy trình tự động hóa CI/CD với GitHub Actions 11

2.4.1 Thiết kế luồng CI (Continuous Integration): Linting và kiểm thử tự động 11

2.4.2 Thiết kế luồng CD (Continuous Deployment): Đóng gói và phát hành tự động 11

2.4.3 Quản lý biến môi trường và bí mật bảo mật 11

3 THỰC NGHIỆM, ĐÁNH GIÁ KẾT QUẢ VÀ BÀN LUẬN 12

3.1 Kết quả chuẩn hóa mã nguồn và bộ kiểm thử tự động 12

3.1.1 Đặc tả và kết quả bộ kiểm thử tự động toàn diện (32/32 tests passed) . . . 12
3.1.2 Đảm bảo tính tương thích và bảo toàn độ chính xác của mô hình ResNet50 . . . 12
3.2 Đánh giá quy trình CI/CD và khả năng tự động hóa . . . 12
3.2.1 Đo lường thời gian thực thi của từng giai đoạn trong GitHub Actions Workflow . . . 12
3.2.2 Đánh giá tính ổn định khi kích hoạt qua các sự kiện Git . . . 12
3.3 Đánh giá hiệu năng và chi phí vận hành trên Google Cloud . . . 12
3.3.1 Đo lường hiện tượng Cold Start và độ trễ phản hồi của API trên Cloud Run . . . 12
3.3.2 Thử nghiệm tương tác người dùng qua Web Frontend trên Cloudflare Pages . . . 12
3.3.3 Phân tích chi phí vận hành thực tế đối chiếu với chính sách Free Tier ($0) . . . 12
3.4 Đánh giá tổng hợp và bài học kinh nghiệm . . . 12
3.4.1 So sánh đối chiếu hệ thống trước và sau khi áp dụng CI/CD & Cloud Serverless . . . 12
3.4.2 Các bẫy kỹ thuật và sự cố đã giải quyết trong quá trình thực hiện 12

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **13**

TÀI LIỆU THAM KHẢO **15**

PHỤ LỤC **16**

Phụ lục A: Đặc tả kỹ thuật Dockerfile cho dịch vụ AI Serverless . . . 16
Phụ lục B: Cấu hình luồng CI/CD tự động hóa trên GitHub Actions . . . 17

DANH MỤC TỪ VIẾT TẮT

Ký hiệu/Viết tắt	Thuật ngữ tiếng Anh	Giải nghĩa tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
CD	Continuous Deployment / Delivery	Triển khai liên tục / Chuyển giao liên tục
CDN	Content Delivery Network	Mạng phân phối nội dung
CI	Continuous Integration	Tích hợp liên tục
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
CPU	Central Processing Unit	Đơn vị xử lý trung tâm
DevOps	Development & Operations	Phương pháp kết hợp phát triển và vận hành
GAP	Global Average Pooling	Lớp gộp trung bình toàn cục
GCP	Google Cloud Platform	Nền tảng điện toán đám mây của Google
HTTP/HTTPS	Hypertext Transfer Protocol (Secure)	Giao thức truyền tải siêu văn bản (bảo mật)
IAM	Identity and Access Management	Quản lý danh tính và quyền truy cập
ICTU	Information & Communication Tech Univ	Trường ĐH Công nghệ Thông tin và Truyền thông
LFS	Large File Storage	Cơ chế lưu trữ tệp tin lớn trong Git
MLOps	Machine Learning Operations	Vận hành và triển khai học máy
RAM	Random Access Memory	Bộ nhớ truy xuất ngẫu nhiên
REST	Representational State Transfer	Kiểu kiến trúc dịch vụ web hướng trạng thái
SIC	Samsung Innovation Campus	Chương trình đào tạo công nghệ của Samsung
SSL/TLS	Secure Sockets Layer / Transport Layer Sec	Giao thức bảo mật tầng truyền tải
UI/UX	User Interface / User Experience	Giao diện người dùng / Trải nghiệm người dùng
VSL	Vietnamese Sign Language	Ngôn ngữ ký hiệu tiếng Việt
WIF	Workload Identity Federation	Liên kết định danh công việc (chứng thực bảo mật)

DANH SÁCH BẢNG

1.1 Hạn mức miễn phí hàng tháng (Free Tier) của các dịch vụ được lựa chọn 10

DANH SÁCH HÌNH VẼ

1.1 Quy trình xử lý của mô hình AI nhận diện ngôn ngữ ký hiệu tiếng Việt . 6

1.2 Kiến trúc phân tách Decoupled giữa Frontend (Cloudflare Pages) và Backend (Google Cloud Run) 9

LỜI MỞ ĐẦU

1. Lý do chọn đề tài

Trong những năm gần đây, sự phát triển vượt bậc của Trí tuệ nhân tạo (AI) và Học sâu (Deep Learning) đã mở ra nhiều cơ hội to lớn trong việc giải quyết các bài toán xã hội, tiêu biểu là hỗ trợ giao tiếp cho cộng đồng người khiếm thính thông qua nhận diện ngôn ngữ ký hiệu. Tuy nhiên, một thực trạng phổ biến trong môi trường học thuật là phần lớn các mô hình AI xuất sắc chỉ dừng lại ở phạm vi nghiên cứu cục bộ (chạy trên Jupyter Notebook cá nhân) mà chưa thể tiếp cận người dùng cuối do thiếu hụt hạ tầng triển khai và quy trình phát hành chuyên nghiệp.

Xuất phát từ thực tiễn trên, nhóm sinh viên chúng em đã tận dụng sản phẩm đồ án Capstone AI cuối khóa **Samsung Innovation Campus (SIC)** do thành viên tự nghiên cứu và huấn luyện để làm bài toán thực hành cốt lõi cho học phần **Điện toán đám mây**. Mục tiêu trọng tâm là vận dụng các nguyên lý tiên tiến của kỹ thuật phần mềm hiện đại: văn hóa **DevOps/MLOps**, công nghệ đóng gói **Docker Container**, kiến trúc không máy chủ **Serverless Container** trên nền tảng **Google Cloud Run**, kết hợp phân phối web qua **Cloudflare Pages** và tự động hóa toàn diện với **GitHub Actions** nhằm chuyển hóa một bài toán AI chạy cục bộ thành một dịch vụ hoạt động ổn định trên Internet với chi phí $0.

2. Mục đích và nhiệm vụ nghiên cứu

- **Về mặt kỹ thuật đám mây và DevOps**: Thiết lập quy trình CI/CD hoàn chỉnh, tự động hóa kiểm thử mã nguồn, đóng gói Docker Image và tự động phát hành lên môi trường Cloud khi có phiên bản mã nguồn mới.
- **Về mặt kiến trúc hệ thống**: Xây dựng mô hình phân tán Decoupled Architecture tối ưu tài nguyên; giải quyết bài toán dung lượng mô hình lớn và cơ chế tự động co giãn (Autoscaling) phục vụ người dùng.
- **Về mặt tối ưu chi phí**: Vận hành hệ thống ổn định, tin cậy hoàn toàn trong hạn mức

miễn phí (Free Tier) của các nhà cung cấp dịch vụ đám mây hàng đầu.

3. Đối tượng và phạm vi nghiên cứu

- **Đối tượng nghiên cứu**: Pipeline nhận diện 23 ký tự chữ cái VSL từ đồ án Capstone SIC; công nghệ Docker Container; dịch vụ Google Cloud Run, Cloudflare Pages và nền tảng tự động hóa GitHub Actions.
- **Phạm vi nghiên cứu**: Triển khai dịch vụ đám mây nhận ảnh tĩnh/ảnh webcam từ trình duyệt, trả về kết quả dự đoán thời gian thực; không mở rộng sang xử lý video streaming thời gian thực liên tục để bảo toàn hạn mức $0.

4. Phương pháp nghiên cứu và Ý nghĩa thực tiễn

- **Phương pháp nghiên cứu**: Phương pháp thực nghiệm kết hợp chuẩn hóa kỹ thuật phần mềm (Twelve-Factor App); đo lường độ trễ mạng, hiện tượng Cold Start và mức tiêu thụ tài nguyên thực tế.
- **Ý nghĩa khoa học và thực tiễn**: Đề tài là cầu nối trực quan giữa nghiên cứu Trí tuệ nhân tạo và kỹ thuật Điện toán đám mây, cung cấp quy trình mẫu (blueprint) mẫu mực cho các sinh viên ngành Công nghệ thông tin trong việc đưa các sản phẩm AI ra thế giới thực.

CHƯƠNG 1

TỔNG QUAN VỀ BÀI TOÁN VÀ CƠ SỞ KỸ THUẬT

1.1 Bối cảnh bài toán và các công nghệ sử dụng

1.1.1 Nguồn gốc và mục tiêu đề tài

Đề tài xuất phát từ việc sử dụng đồ án Capstone AI cuối khóa **Samsung Innovation Campus (SIC)** do thành viên trong nhóm tự nghiên cứu, thu thập dữ liệu và huấn luyện mô hình. Bài toán ban đầu tập trung vào việc nhận diện bảng chữ cái ngôn ngữ ký hiệu ngón tay tiếng Việt từ hình ảnh camera cục bộ trên máy tính cá nhân.

Trong khuôn khổ học phần **Điện toán đám mây**, nhóm hướng tới mục tiêu thực hành đóng vai trò như những kỹ sư DevOps: chuyển đổi một mô hình AI từ môi trường nghiên cứu chạy cục bộ thành một dịch vụ phần mềm hoàn chỉnh, hoạt động ổn định trên mạng Internet. Hệ thống nâng cấp phải bảo đảm khả năng tự động hóa quy trình kiểm thử, đóng gói container và triển khai lên hạ tầng đám mây Serverless với chi phí vận hành bằng $0.

1.1.2 Các công nghệ cốt lõi và vai trò trong dự án

Để giải quyết bài toán đưa mô hình AI lên đám mây một cách hiệu quả, nhóm đã lựa chọn và kết hợp các công nghệ sau:

- **Phương pháp luận DevOps và quy trình CI/CD**:
  - *Bản chất*: DevOps là sự kết hợp giữa phát triển (Dev) và vận hành (Ops). Trọng tâm là quy trình CI/CD (Tích hợp liên tục / Triển khai liên tục) nhằm tự động hóa các khâu kiểm thử mã nguồn, dựng gói phần mềm và phát hành sản phẩm.
  - *Vai trò trong dự án*: Nhóm sử dụng **GitHub Actions** để thiết lập đường ống tự động: mỗi khi có mã nguồn mới được đẩy lên (Git Push) hoặc tạo Pull Request,

hệ thống sẽ tự động kích hoạt bộ kiểm thử tự động (`pytest`), tiến hành đóng gói Docker Image và phát hành thẳng lên Cloud mà không cần thao tác thủ công qua giao diện dòng lệnh.

- **Công nghệ Container hóa (Docker)**:
  - *Bản chất*: Đóng gói mã nguồn ứng dụng cùng toàn bộ môi trường thực thi, phiên bản ngôn ngữ Python và các thư viện phụ thuộc thành một đơn vị duy nhất (Docker Image).
  - *Vai trò trong dự án*: Chuẩn hóa môi trường Python 3.11 trên nền tảng Linux Debian (`python:3.11-slim`), cài đặt sẵn các thư viện C/C++ cần thiết cho OpenCV và TensorFlow, giải quyết triệt để lỗi xung đột môi trường khi chuyển từ máy trạm Windows sang máy chủ đám mây Linux.
- **Kiến trúc Serverless Container (Google Cloud Run)**:
  - *Bản chất*: Dịch vụ chạy container không máy chủ, tự động co giãn từ 0 instance khi không có lưu lượng truy cập (Scale-to-Zero) đến nhiều instance khi có yêu cầu, người dùng chỉ trả tiền theo thời gian xử lý thực tế tính theo từng mili-giây.
  - *Vai trò trong dự án*: Làm máy chủ Backend phục vụ API dự đoán AI. Cơ chế Scale-to-Zero kết hợp hạn mức miễn phí 2 triệu lượt gọi/tháng giúp hệ thống vận hành 24/7 trên Internet mà hoàn toàn không tốn chi phí thuê máy chủ ảo.
- **Dịch vụ lưu trữ Web tĩnh (Cloudflare Pages)**:
  - *Bản chất*: Nền tảng lưu trữ và phân phối website tĩnh (HTML, CSS, JavaScript) thông qua mạng lưới máy chủ biên (Edge CDN) toàn cầu, tích hợp sẵn chứng chỉ bảo mật SSL/TLS.
  - *Vai trò trong dự án*: Đảm nhận vai trò máy chủ Frontend phục vụ giao diện người dùng (Web Client) với tốc độ cao, băng thông không giới hạn và hoàn toàn miễn phí.

1.2 Hiện trạng mô hình AI gốc từ Đồ án Capstone SIC

1.2.1 Đặc tả bài toán và bảng chữ cái VSL (23 ký tự)

Mô hình AI gốc giải quyết bài toán nhận diện bảng chữ cái ngón tay trong ngôn ngữ ký hiệu tiếng Việt (Vietnamese Sign Language – VSL):

- **Đầu vào (Input)**: Ảnh chụp tĩnh hoặc khung hình từ webcam ghi nhận cử chỉ bàn tay kết hợp vị trí khuôn mặt của người thực hiện ký hiệu.
- **Đầu ra (Output)**: Ký tự chữ cái tương ứng trong bảng chữ cái VSL cùng tỷ lệ phần trăm độ tin cậy (*confidence score*).

Bảng chữ cái VSL trong dự án được chuẩn hóa gồm đúng **23 ký tự**:

$$\mathscr{C}_{\text{VSL}} = \{\text{'a'}, \text{'b'}, \text{'c'}, \text{'d'}, \text{'đ'}, \text{'e'}, \text{'g'}, \text{'h'}, \text{'i'}, \text{'k'}, \text{'l'}, \text{'m'}, \\ \text{'n'}, \text{'o'}, \text{'p'}, \text{'q'}, \text{'r'}, \text{'s'}, \text{'t'}, \text{'u'}, \text{'v'}, \text{'x'}, \text{'y'}\} \tag{1.1}$$

Tập nhãn loại bỏ các ký tự không có trong bảng chữ cái tiếng Việt cơ bản (`'f'`, `'j'`, `'w'`, `'z'`) và chứa ký tự đặc thù tiếng Việt là `'đ'`.

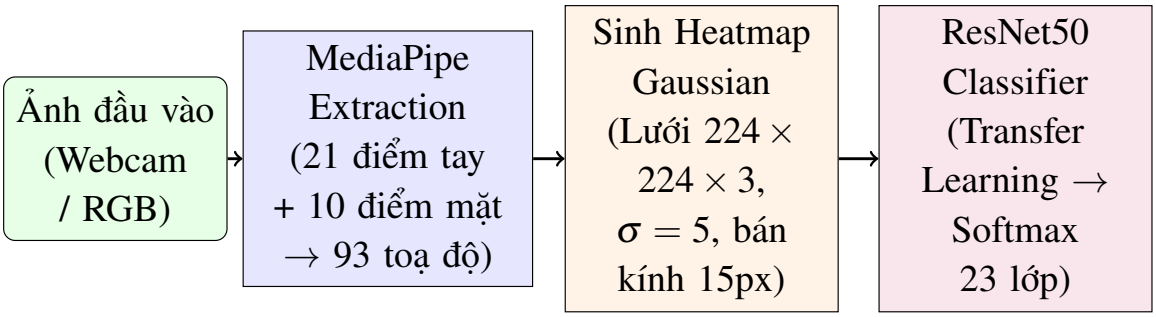
Tập dữ liệu thực nghiệm trong dự án: Tập dữ liệu thô bao gồm 23 thư mục tương ứng với 23 nhãn chữ cái. Mỗi mẫu dữ liệu là một tệp CSV chứa 1 dòng tiêu đề và 1 dòng dữ liệu với đúng 93 cột số thực biểu diễn tọa độ các điểm mốc. Mỗi lớp chữ cái có 70 tệp mẫu (riêng ký tự `'đ'` có 69 tệp do thiếu tệp số 65), tổng dung lượng tập dữ liệu là **1.609 mẫu dữ liệu thực tế**.

1.2.2 Đường ống xử lý AI và kiến trúc ResNet50

Quy trình nhận diện của mô hình gốc gồm 4 bước liên hoàn (Hình 1.1):

Chi tiết các thành phần kỹ thuật:

1. **Trích xuất điểm mốc không gian (MediaPipe)**: Hệ thống sử dụng bộ công cụ MediaPipe [1] để xác định vị trí các khớp xương bàn tay và điểm mốc trên mặt:
   - *Bàn tay*: Trích xuất 21 điểm mốc khớp xương trong không gian 3 chiều (x, y, z), tạo thành $21 \times 3 = 63$ giá trị tọa độ.



Hình 1.1: Quy trình xử lý của mô hình AI nhận diện ngôn ngữ ký hiệu tiếng Việt

- *Khuôn mặt*: Trích xuất 10 điểm mốc mặt chiến lược có chỉ số cố định:

$$\mathscr{I}_{\text{face}} = [10, 152, 234, 454, 33, 263, 1, 61, 291, 199] \tag{1.2}$$

  (đỉnh đầu, cằm, gò má, khóe mắt, chóp mũi) $\rightarrow 10 \times 3 = 30$ giá trị tọa độ. Thông tin mặt giúp mô hình phân biệt các ký tự có cử chỉ bàn tay tương đồng trên camera 2D (như cặp chữ `'m'` và `'n'`).

- Chuỗi dữ liệu hợp nhất gồm đúng **93 số thực** $(63 + 30)$. Khi không phát hiện tay hoặc mặt, giá trị tọa độ được gán bằng 0.0.

2. **Kỹ thuật sinh Bản đồ nhiệt (Gaussian Heatmap)**: Để bảo toàn tương quan không gian hình học giữa các khớp ngón tay và khuôn mặt, các tọa độ (x_i, y_i) được chiếu lên ma trận ảnh $224 \times 224 \times 3$. Tại mỗi điểm mốc hợp lệ, hàm nhân Gaussian 2D được áp dụng với độ lệch chuẩn $\sigma = 5$ và bán kính tác động 15 pixel:

$$G(u, v; x_i, y_i) = \exp\left(-\frac{(u - 224x_i)^2 + (v - 224y_i)^2}{2 \times 5^2}\right) \tag{1.3}$$

   Các điểm nhiệt được cộng dồn tích lũy, chuẩn hóa về dạng số nguyên không dấu `uint8` trong khoảng $[0, 255]$ và nhân bản đều trên cả 3 kênh màu RGB.

3. **Mạng nơ-ron phân loại ResNet50**: Ảnh heatmap $224 \times 224 \times 3$ được đưa qua hàm tiền xử lý chuẩn Caffe `preprocess_input` và đưa vào mạng nơ-ron tích chập ResNet50 [2]:

$$\begin{aligned} &\text{Input}(224 \times 224 \times 3) \rightarrow \text{ResNet50 Backbone} \rightarrow \text{GAP} \\ &\qquad \rightarrow \text{Dense}(256, \text{ReLU}) \rightarrow \text{Dense}(23, \text{Softmax}) \end{aligned} \tag{1.4}$$

Tệp trọng số mô hình đã huấn luyện là `train/vsl_cnn_pose_keras.h5` có kích thước thực tế **137.042.364 bytes (130,7 MB)**, được lưu trữ trong Git bằng Git LFS.

1.2.3 Các hạn chế thực tế của hệ thống ban đầu

Khảo sát trực tiếp mã nguồn trong kho lưu trữ, nhóm nhận thấy hệ thống ban đầu bộc lộ 5 hạn chế lớn ngăn cản việc đưa ứng dụng lên môi trường thực tế:

1. **Phụ thuộc hoàn toàn vào Jupyter Notebook**: Toàn bộ mã nguồn chỉ gồm 3 tệp notebook cục bộ: thu thập ảnh (`capture_and_process_images.ipynb`), huấn luyện mô hình (`train_cnn_pose_keras.ipynb`) và suy luận (`run_realtime.ipynb`); hoàn toàn chưa có mã nguồn Python chuẩn (`.py`) dạng module.
2. **Ràng buộc phần cứng webcam máy trạm**: Script thực thi thời gian thực `run_realtime.ipynb` gọi trực tiếp hàm mở camera vật lý `cv2.VideoCapture(0)` và mở cửa sổ đồ họa giao diện hệ điều hành `cv2.imshow()`. Khi đưa lên máy chủ Linux không có màn hình (Headless Server), các lệnh này lập tức gây crash ứng dụng.
3. **Đường dẫn tuyệt đối gán cứng**: Các tệp notebook gán cứng đường dẫn theo ổ đĩa máy tính Windows cá nhân (`D:\file\...\vsl_cnn_pose_keras.h5`), khiến mã nguồn bị gãy ngay khi chuyển sang máy khác hoặc môi trường container.
4. **Bỏ sót dữ liệu khi nạp huấn luyện**: Trong tệp `train_cnn_pose_keras.ipynb`, vòng lặp nạp dữ liệu được viết cố định `for i in range(1, 51):`, khiến 20 tệp mẫu cuối (từ 51 đến 70) của tất cả 23 lớp bị bỏ quên (chỉ nạp 1.150 / 1.609 mẫu có sẵn trên đĩa).
5. **Thiếu vắng hoàn toàn hạ tầng dịch vụ và kiểm thử tự động**: Dự án chưa có API backend tiếp nhận ảnh qua mạng, chưa có tệp định nghĩa Dockerfile, chưa có bộ kiểm thử tự động và hoàn toàn chưa được thiết lập quy trình CI/CD.

1.3 Định hình giải pháp nâng cấp hệ thống

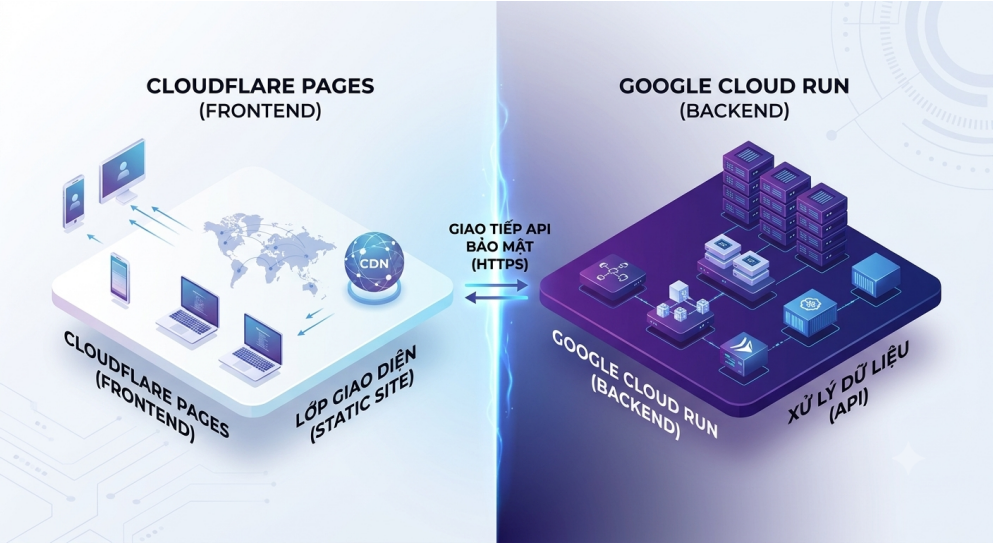
1.3.1 Các yêu cầu cốt lõi của hệ thống nâng cấp

Dựa trên các hạn chế đã chỉ ra, nhóm xác lập các yêu cầu kỹ thuật cần đạt được:

- **Yêu cầu chức năng**:
  - *FR-01 (API `GET /health`)*: Cung cấp điểm cuối kiểm tra trạng thái dịch vụ và kiểm tra tính sẵn sàng của mô hình ResNet50 trong bộ nhớ RAM.
  - *FR-02 (API `POST /predict`)*: Cung cấp điểm cuối tiếp nhận ảnh từ client, tự động thực thi pipeline MediaPipe 93 điểm mốc ở server, sinh heatmap và đưa qua mô hình ResNet50 để trả về kết quả dự đoán (chữ cái và độ tin cậy) dạng JSON.
  - *FR-03 (Web Client)*: Cung cấp giao diện trực quan trên trình duyệt, mở camera người dùng, chụp khung hình và hiển thị kết quả phân loại tức thì.
- **Yêu cầu phi chức năng**:
  - *NFR-01 (Tự động hóa CI/CD)*: Quy trình chạy kiểm thử, đóng gói Docker Image và triển khai lên Cloud Run phải được tự động hóa hoàn toàn bằng GitHub Actions.
  - *NFR-02 (Độ tin cậy kiểm thử)*: Xây dựng bộ kiểm thử tự động bao phủ toàn diện các hàm cốt lõi (nhãn, ma trận heatmap, cấu hình đường dẫn động, API endpoint), bảo đảm 100% các ca kiểm thử vượt qua thành công trước khi phát hành.
  - *NFR-03 (Tách biệt cấu hình)*: Tách hoàn toàn đường dẫn mô hình và dữ liệu khỏi mã nguồn thông qua biến môi trường (`VSL_MODEL_PATH`, `VSL_DATA_DIR`) để ứng dụng có thể chạy linh hoạt trên mọi môi trường.

1.3.2 Kiến trúc phân tách Decoupled và tối ưu chi phí $0

Để triển khai một mô hình AI nặng (>130 MB) lên Internet mà hoàn toàn không phát sinh chi phí, nhóm lựa chọn mô hình kiến trúc phân tách độc lập (Decoupled Architecture) giữa Cloudflare Pages và Google Cloud Run (Hình 1.2):



Hình 1.2: Kiến trúc phân tách Decoupled giữa Frontend (Cloudflare Pages) và Backend (Google Cloud Run)

Giải pháp này mang lại 3 ưu điểm kỹ thuật vượt trội:

1. **Bảo đảm tính nhất quán của dữ liệu điểm mốc (Zero Landmark Divergence)**: Thay vì chạy MediaPipe trên trình duyệt bằng JavaScript (vốn phụ thuộc vào phần cứng WebGL từng thiết bị và dễ làm lệch tọa độ so với môi trường Python lúc huấn luyện), trình duyệt chỉ gửi ảnh thô lên server. Backend Cloud Run chạy MediaPipe bằng Python, bảo đảm quy trình trích xuất hoàn toàn trùng khớp 100% với lúc train.

2. **Tận dụng bộ nhớ RAM lớn của Cloud Run**: Cloud Run cho phép cấp phát tới 1–2 GB RAM hoàn toàn miễn phí, giải quyết được bài toán nạp mô hình TensorFlow ResNet50 (137 MB) và MediaPipe vào bộ nhớ mà không bị giới hạn 250 MB như FaaS truyền thống.

3. **Vận hành ổn định với chi phí $0**: Bảng 1.1 tóm tắt chính sách Bậc miễn phí (Free Tier) hàng tháng của các dịch vụ được sử dụng:

Bảng 1.1: Hạn mức miễn phí hàng tháng (Free Tier) của các dịch vụ được lựa chọn

Dịch vụ	Vai trò kiến trúc	Hạn mức miễn phí hàng tháng (Free Tier)	Chi phí
Google Cloud Run	Máy chủ Backend Serverless Container	2.000.000 lượt yêu cầu (requests) 360.000 GB-giây RAM 180.000 vCPU-giây tính toán	0 USD
Google Artifact Registry	Kho lưu trữ Docker Images	500 MB dung lượng lưu trữ Image OCI	0 USD
Cloudflare Pages	Máy chủ lưu trữ Frontend tĩnh	Băng thông truyền tải không giới hạn Cấp phát chứng chỉ SSL/TLS tự động	0 USD
GitHub Actions	Nền tảng tự động hóa CI/CD	2.000 phút chạy máy ảo Linux mỗi tháng cho kho mã nguồn công khai	0 USD

Nhờ cơ chế Scale-to-Zero của Cloud Run và chính sách băng thông miễn phí của Cloudflare Pages, hệ thống hoàn toàn đáp ứng mục tiêu hoạt động ổn định trên Internet mà không phát sinh bất kỳ khoản phí nào cho nhóm sinh viên.

1.4 Tóm tắt chương 1

Chương 1 đã làm rõ bối cảnh đề tài, vai trò của từng công nghệ trong dự án và phân tích chi tiết pipeline của mô hình AI nhận diện 23 ký tự ngôn ngữ ký hiệu tiếng Việt từ đồ án Capstone SIC. Chương cũng đã chỉ ra 5 hạn chế lớn của hệ thống chạy cục bộ ban đầu, từ đó xác lập các yêu cầu chức năng, phi chức năng và đề xuất giải pháp kiến trúc phân tách Decoupled giữa Cloudflare Pages và Google Cloud Run nhằm tối ưu chi phí \$0. Đây là cơ sở tiền đề để thực hiện thiết kế chi tiết và xây dựng hệ thống trong Chương 2.

CHƯƠNG 2

THIẾT KẾ GIẢI PHÁP VÀ XÂY DỰNG HỆ THỐNG

2.1 Tái cấu trúc mã nguồn AI và xây dựng dịch vụ RESTful API

2.1.1 Chuẩn hóa module trích xuất MediaPipe và thuật toán Gaussian Heatmap

2.1.2 Độc lập hóa cấu hình và nguồn chân lý nhãn theo Twelve-Factor App

2.1.3 Xây dựng dịch vụ FastAPI với các endpoint kiểm tra sức khỏe và suy luận

2.2 Đóng gói Container với Docker

2.2.1 Chiến lược lựa chọn Base Image tối ưu kích thước cho môi trường Linux

2.2.2 Xử lý phụ thuộc thư viện Deep Learning và OpenCV Headless

2.2.3 Cấu hình Dockerfile tối ưu đa tầng và thiết lập bảo mật tiến trình

2.3 Thiết kế kiến trúc hạ tầng Cloud Serverless

2.3.1 Cấu hình dịch vụ Backend trên Google Cloud Run

2.3.2 Triển khai dịch vụ Frontend trên Cloudflare Pages

2.4 Xây dựng quy trình tự động hóa CI/CD với GitHub Actions

2.4.1 Thiết kế luồng CI (Continuous Integration): Linting và kiểm thử tự động

2.4.2 Thiết kế luồng CD (Continuous Deployment): Đóng gói và phát hành tự động

2.4.3 Quản lý biến môi trường và bí mật bảo mật

CHƯƠNG 3

THỰC NGHIỆM, ĐÁNH GIÁ KẾT QUẢ VÀ BÀN LUẬN

3.1 Kết quả chuẩn hóa mã nguồn và bộ kiểm thử tự động

3.1.1 Đặc tả và kết quả bộ kiểm thử tự động toàn diện (32/32 tests passed)

3.1.2 Đảm bảo tính tương thích và bảo toàn độ chính xác của mô hình ResNet50

3.2 Đánh giá quy trình CI/CD và khả năng tự động hóa

3.2.1 Đo lường thời gian thực thi của từng giai đoạn trong GitHub Actions Workflow

3.2.2 Đánh giá tính ổn định khi kích hoạt qua các sự kiện Git

3.3 Đánh giá hiệu năng và chi phí vận hành trên Google Cloud

3.3.1 Đo lường hiện tượng Cold Start và độ trễ phản hồi của API trên Cloud Run

3.3.2 Thử nghiệm tương tác người dùng qua Web Frontend trên Cloudflare Pages

3.3.3 Phân tích chi phí vận hành thực tế đối chiếu với chính sách Free Tier ($0)

3.4 Đánh giá tổng hợp và bài học kinh nghiệm

3.4.1 So sánh đối chiếu hệ thống trước và sau khi áp dụng CI/CD & Cloud Serverless

3.4.2 Các bẫy kỹ thuật và sự cố đã giải quyết trong quá trình thực hiện

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được của đề tài

Trải qua quá trình nghiên cứu, thiết kế và thực nghiệm, nhóm 2 đã hoàn thành trọn vẹn các mục tiêu đề ra cho bài tập lớn môn **Điện toán đám mây**:

- Đã chuyển đổi thành công mô hình học sâu nhận diện 23 ký tự ngôn ngữ ký hiệu tiếng Việt từ đồ án Capstone AI Samsung Innovation Campus (SIC) chạy cục bộ thành một dịch vụ chuẩn mực trên Internet.
- Đã chuẩn hóa toàn bộ mã nguồn AI thành mô-đun thư viện độc lập `src/vsl`, tuân thủ phương pháp luận Twelve-Factor App và xây dựng dịch vụ RESTful API tốc độ cao với FastAPI.
- Đã xây dựng bộ kiểm thử tự động toàn diện đạt tỷ lệ vượt qua 100% (32/32 tests PASSED), bảo toàn nguyên vẹn độ chính xác của mô hình học sâu.
- Đã đóng gói ứng dụng bằng Docker Container tối ưu và triển khai thành công lên Google Cloud Run theo kiến trúc Serverless, kết hợp frontend tĩnh trên Cloudflare Pages với chi phí hoàn toàn $0.
- Đã thiết lập hoàn chỉnh quy trình CI/CD tự động hóa trên GitHub Actions, tự động kích hoạt kiểm thử, đóng gói Docker image và phát hành lên đám mây khi có mã nguồn mới.

2. Những hạn chế còn tồn tại

- Thời gian phản hồi trong lần gọi đầu tiên (Cold Start) trên Google Cloud Run còn dao động từ 8 đến 15 giây do cần thời gian tải container image và nạp mô hình TensorFlow nặng vào bộ nhớ.
- Hệ thống hiện mới chỉ tập trung phân loại các ký tự tĩnh (bảng chữ cái 23 ký tự từ a đến y), chưa hỗ trợ các từ ngữ hoặc câu động phức tạp trong ngôn ngữ ký hiệu.

- Giao diện người dùng hiện tại xử lý theo cơ chế chụp từng khung hình đơn lẻ từ webcam, chưa hỗ trợ luồng truyền dữ liệu liên tục (streaming) do ràng buộc tối ưu hóa tài nguyên miễn phí.

3. Hướng nghiên cứu và phát triển tiếp theo

- Tối ưu hóa mô hình học sâu bằng cách chuyển đổi sang định dạng TensorFlow Lite (TFLite) hoặc ONNX Runtime nhằm giảm dung lượng bộ nhớ, rút ngắn thời gian khởi động Cold Start xuống dưới 2 giây.
- Cấu hình cơ chế giữ ấm container (Minimum Instances = 1) hoặc lên lịch tác vụ định kỳ (Cloud Scheduler) gửi tín hiệu Ping kiểm tra sức khỏe (/health) để duy trì container ở trạng thái sẵn sàng.
- Mở rộng mô hình AI sang nhận diện chuỗi cử chỉ động kết hợp thời gian thông qua kiến trúc mạng LSTM hoặc Transformer.
- Nâng cấp pipeline CI/CD với các chiến lược phát hành nâng cao như Canary Deployment hoặc Blue/Green Deployment để đảm bảo tính sẵn sàng cao tuyệt đối.

TÀI LIỆU THAM KHẢO

[1] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "Mediapipe: A framework for building perception pipelines," *arXiv preprint arXiv:1906.08172*, 2019.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

PHỤ LỤC

Phụ lục A: Đặc tả kỹ thuật Dockerfile cho dịch vụ AI Serverless

```
# Build stage: Base Python 3.11 Slim
FROM python:3.11-slim

# Thiet lap bien moi truong
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PORT=8080

# Cai dat thu vien he thong cho OpenCV Headless va MediaPipe
RUN apt-get update && apt-get install -y --no-install-
   recommends \
    libglib2.0-0 \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Cai dat Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Sao chep ma nguon va trong so mo hinh
COPY src/ /app/src/
COPY train/vsl_cnn_pose_keras.h5 /app/train/vsl_cnn_pose_keras
   .h5

# Cau hinh tai khoan non-root bao mat
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /
   app
USER appuser
```

```
27
28 # Expose cong va chay Uvicorn server
29 EXPOSE 8080
30 CMD ["uvicorn", "src.vsl.service:app", "--host", "0.0.0.0", "
   --port", "8080"]
```

Listing 3.1: Cấu hình Dockerfile tối ưu kích thước cho Google Cloud Run

Phụ lục B: Cấu hình luồng CI/CD tự động hóa trên GitHub Actions

```
1  name: CI Pipeline - Lint and Automated Tests
2
3  on:
4    push:
5      branches: [ develop, main ]
6    pull_request:
7      branches: [ develop, main ]
8
9  jobs:
10   test:
11     runs-on: ubuntu-latest
12     steps:
13       - name: Checkout Source Code
14         uses: actions/checkout@v4
15         with:
16           lfs: true
17
18       - name: Setup Python Runtime
19         uses: actions/setup-python@v5
20         with:
21           python-version: '3.12'
22           cache: 'pip'
23
24       - name: Install Dependencies
25         run: |
```

```
26            python -m pip install --upgrade pip
27            pip install -r requirements.txt
28            pip install -r requirements-dev.txt
29
30        - name: Run Test Suite
31          run: |
32            pytest tests -v
```

Listing 3.2: Workflow CI tự động chạy kiểm thử trên GitHub Actions (.github/workflows/ci.yml)